FengHuang Tensor Addressable Bridge

Implementation and Evaluation Report*

Anton Melnychuk
Yale University

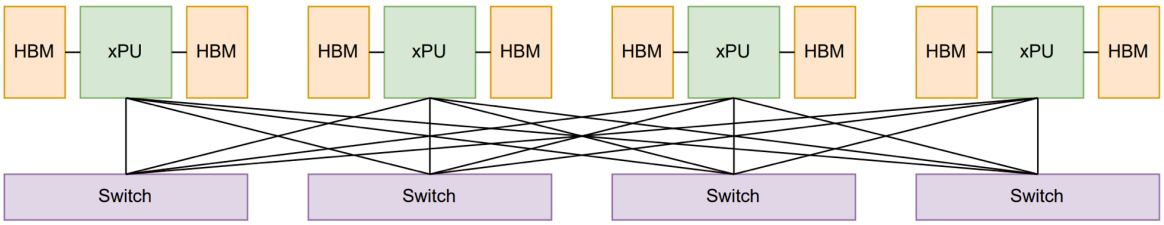
April 2026

Abstract

This report builds on a FengHuang [1] paper from Microsoft Research Asia that presents a vision for a novel AI infrastructure design that has been initially validated through inference simulations on state-of-the-art large language models. We walk through the full hardware-software co-design flow that an industrial architecture team would follow before the tape-out. We implement RTL, software components (prefetch simulations, speculative MoE prototype), and elaborate on physical integration. We conclude that the proposed "fully decoupled" and "vendor agnostic" memory design is physically infeasible at the proposed scale, given vendor technologies available today. Instead, we outline a "hybrid" implementation and discuss remaining challenges as the realistic path forward.

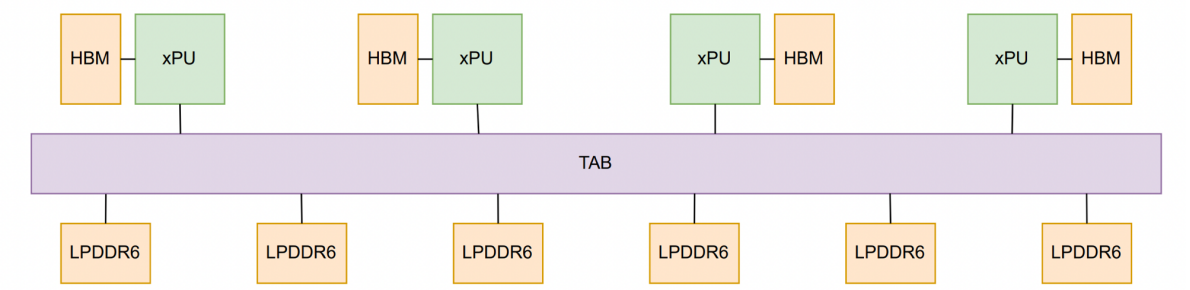
Introduction

FengHuang [1] features a multitier shared-memory architecture combining high-speed local memory with centralized disaggregated remote memory, enhanced by active tensor paging and near-memory compute for tensor operations. Simulations demonstrate that FengHuang achieves up to 93% local memory capacity reduction, 50% GPU compute savings, and 16× to 70× faster interGPU communication compared to conventional GPU scaling. To achieve this, FengHuang proposes a custom GPU-extendable-ISA silicon called the *Tensor Addressable Bridge* (TAB): a shared remote-memory fabric that connects a cluster of GPU-class accelerators (xPUs) to a pool of LPDDR6 DRAM over SerDes links, targeting 4.8 TB/s aggregate bandwidth per GPU. The core idea is to apply hardware-software co-design to decouple storage (GPU's local HBM) from compute on an interposer, hiding latency using the proposed prefetch mechanism, allowing a smaller per-GPU HBM footprint (20 GB) while streaming KV-cache on-demand through the TAB crossbar.



(a) In a conventional cluster each xPU owns a full HBM stack and communicates with peers through a multi-tier switch fabric. AllReduce/AllGather traffic crosses the switches at every step.

*CPSC 4261, Prof. Richard Yang, Yale University.



(a) In FengHuang, a shared Tensor Addressable Bridge (TAB) replaces the switch fabric. Per-xPU HBM shrinks to 20 GB; bulk weight storage moves to a pool of LPDDR6 banks [1].

Problem Statement

Before assessing the paper's proposal, it is worth building intuition for why memory bandwidth is so difficult to move across racks and why NVSwitch and Spectrum-X are still widely used.

GPU clusters achieve large HBM bandwidth by stacking five memory dies on top of the GPU using an on-die interposer, a thin slab of silicon that acts as a wiring layer. Each HBM stack uses a 1024-bit parallel bus. To ensure synchronous low-latency delivery the total distance a signal travels is less than 1 mm. Wide buses are easy to route at close range, and no exotic materials are required.

FengHuang proposes moving this memory to a *separate chip* (the TAB) that sits elsewhere on the server board and connects to the xPUs via high-speed serial links (SerDes). The idea is attractive: a separate chip can hold cheap LPDDR6 instead of expensive HBM, and its capacity can be upgraded independently of the xPU. Moreover, modern AI Infra systems tend to advance in FLOPs, while software algorithms, like mixture of experts (MoE), keep the per-token workload roughly constant. As a result, buying more memory to fit a larger model forces companies to *buy a new GPU.*

It is worth noting that the paper is fully aware of the latency penalty this introduces and proposes an explicit hardware-software co-design mitigation. Section 3.2 defines a two-tier memory hierarchy: *xPU Local Memory* (the HBM retained on each GPU) acts as a high-bandwidth L1 cache, while *FengHuang Remote Memory* (LPDDR6 on the TAB) acts as a large, cheap L2. A dedicated *Tensor Prefetcher* runs on a background paging stream, requiring NCCL modifications, moving tensors from remote memory into local HBM ahead of their use by the compute stream and evicting tensors that are no longer needed. Crucially, this prefetching is not speculative: transformer inference follows a statically known execution order, so the prefetcher knows *exactly* which tensors will be needed and when, well ahead of time. In the ideal case, prefetch and compute overlap fully and the remote latency is hidden entirely. This design is sound: the two-tier hierarchy with software-managed prefetch is the correct architectural response to disaggregated memory.

Prefetch Simulation

We reproduced the paper's configuration on the same hardware class the paper used, then built our own simulator [2] to check the prefetch claim and to expose the levers the paper leaves implicit.

Setup. We ran the same model, framework, and workload as the paper: Qwen3-235B-A22B in FP8, SGLang v0.4.10.post2, batch size 8, 4096 prompt tokens, 1024 decode tokens, on the Yale 8×H200 SXM5 cluster (141 GB HBM3e, 4.8 TB/s). We measured two configurations: the standard 8-GPU deployment we use as Baseline8 (≡ 100% throughout), and the FH4 4-GPU shape (FengHuang's target operating point) without TAB. The latter runs 15.7% slower than Baseline8, the gap TAB

is meant to close. To isolate GPU time from Python scheduling overhead we profiled with Nsight Systems and took CUDA graph replay medians from `CUPTI_ACTIVITY_KIND_GRAPH_TRACE`.

We developed an independent simulator [2] to model FengHuang's behaviour using Nsight. Following the paper, we adopt a *lookahead-1* prefetching strategy with prefetch window $w = 1$: while the GPU computes layer n from local HBM, TAB prefetches the layer-n+1 remote-resident weights. The per-layer stall is the part of the prefetch that compute cannot hide, $\max(0,\ t_{\text{prefetch}} - t_{\text{local}})$, and the simulated TPOT is the sum across all $N = 94$ Qwen3-235B-A22B transformer layers.

The shard already fits in HBM, so to exercise TAB at all we apply the capacity rule (20 GB local HBM) to that traffic to split it into a *local* fraction served by HBM and a *remote* fraction served by TAB, parameterised only by the per-GPU shard size. The paper models the prefetch cost as

$$t_{\text{prefetch}} \;=\; \frac{\text{Tensor Size}}{B_{\text{TAB}} \cdot \eta(\text{Tensor Size})}, \tag{1}$$

where $\eta(\cdot) \in (0, 1]$ is an empirical efficiency coefficient (paper Eq. 4.1) that captures the latency-dominated regime for small tensors, similar to NVLink. We set $\eta \equiv 1$, taking B_{TAB} as a hard upper bound; this makes our TPOT estimates conservative-best-case for FengHuang.

Algorithm 1 Lookahead-1 TAB Prefetch Simulator

Inputs: measured t_{GPU}, HBM_BW, shard size W, local cap L, layers N, TAB BW B_{TAB}

1: bytes $\leftarrow t_{\text{GPU}} \cdot$ HBM_BW // *roofline closure*
2: $f_{\text{local}} \leftarrow \min(1,\ L/W)$
3: $t_{\text{local}} \leftarrow (\text{bytes} \cdot f_{\text{local}})/(N \cdot \text{HBM\_BW})$
4: $t_{\text{prefetch}} \leftarrow (\text{bytes} \cdot (1 - f_{\text{local}}))/(N \cdot B_{\text{TAB}})$
5: $t_{\text{stall}} \leftarrow \max(0,\ t_{\text{prefetch}} - t_{\text{local}})$ // *prefetch overlaps compute*
6: **return** $N \cdot (t_{\text{local}} + t_{\text{stall}})$ // *TPOT*

In other words, each layer's TAB transfer (line 4) is overlapped with compute (line 3), and the GPU only stalls by the part the compute cannot hide.

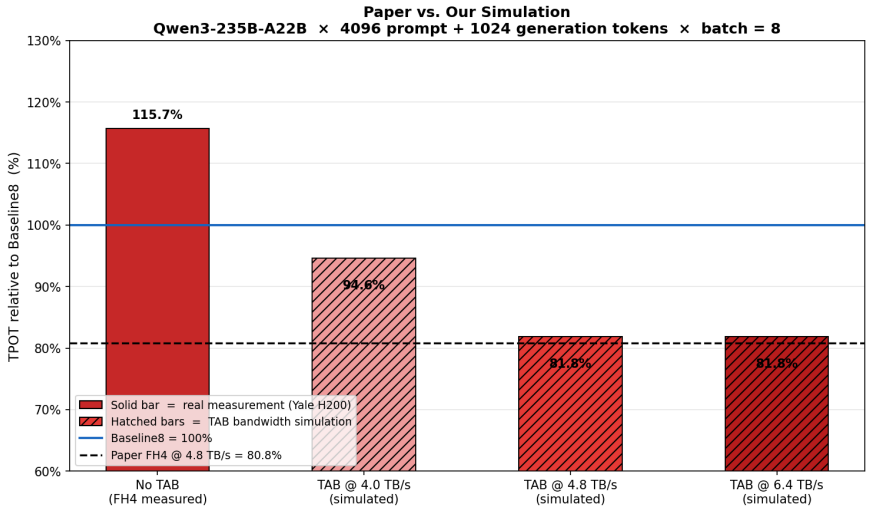


Figure 3: Our simulator's predictions at three TAB bandwidths (dashed reference). At 4.8 TB/s ours lands within 1 % of the paper's reported FH4 result, although at a larger latency scale.

Two unstated tradeoffs. Although results were successfully reproduces, the same simulation, swept across TP degrees, makes two effects visible that the headline numbers initially obscure:

TP dilutes the benefit. The remote fraction is $1 - L/W$. At TP=8 the per-GPU shard is 29 GB and $L = 20$ GB covers 68% of it, leaving only 32% for TAB; even at 4.0 TB/s the prefetch finishes before the GPU does its local share and bandwidth headroom is wasted. At TP=4 the shard is 59 GB, only 34% fits locally, and TAB is the load-bearing path. The paper presents FH4 as a *uniform improvement*; in practice it is a *TP-degree-sensitive* one. At higher TP the TAB bandwidth is overprovisioned and the silicon goes underused; TAB only pays off in a rack-scale "hybrid" deployment whose TP and HBM cap together keep the remote fraction large.

The 20 GB local cap, not the 4.8 TB/s, is the lever. The headline improvement is at least as much a function of *shrinking local HBM* (which forces bytes through TAB) as of TAB bandwidth itself. A 40 GB local cap at the same 4.8 TB/s would roughly halve the remote fraction at TP=4 and erase most of the gain. The paper frames the win as a bandwidth result; it is really a *capacity-vs-bandwidth* substitution, and the cost-efficiency case rests on the assumption that LPDDR6 capacity is genuinely cheaper per GB than the HBM removed, and that those savings outweigh the TAB silicon, package, and control-plane costs the paper does not price in.

Design and RTL Implementation

We built a small synthesizable Verilog model [2] of TAB to sanity-check the paper's protocol: the four operations and their completion semantics. We also aim to make the paper's latency table concrete. The implementation is intentionally minimal as a behavioural reference.

The top module `fenghuang_tab` parameterises `N_XPU=4`, `N_BANKS=4`, `ADDR_W=32`, `DATA_W=512` (a 64 B cache-line) and `TXN_ID_W=8` (256 in-flight transactions per xPU). It exposes the four operations defined in paper Table 3.1: `OP_READ` (220 ns), `OP_WRITE` (90 ns), `OP_WR_ACC` (90 ns, in-memory accumulate used to build AllReduce/ReduceScatter), and `OP_WC_SYNC` (40 ns, write-completion notification). Three helper modules implement the rest: `tab_crossbar` routes xPU requests to bank-side ports, `tab_mem_bank` models LPDDR6 timing and the read-modify-write path for accumulate, and `tab_compl_tracker` tracks outstanding writes per xPU and drives the sync notification mask.

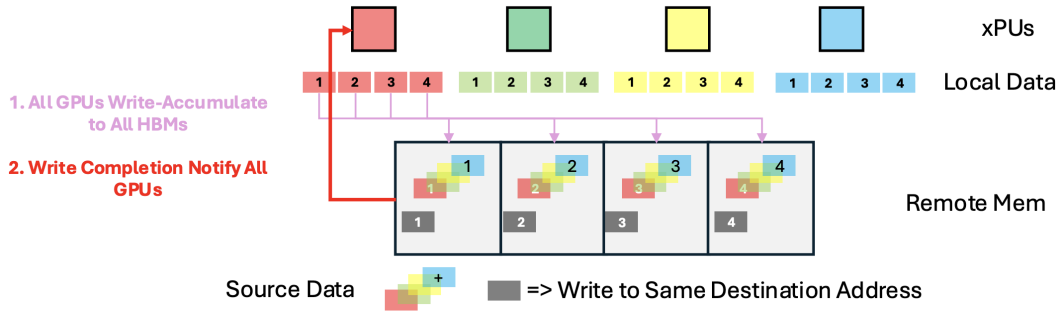


Figure 4: Credit: FengHuang [1]. AllReduce built from the TAB protocol. Step 1 (`OP_WR_ACC`): every xPU writes its local data to the same destination address in remote memory, where the bank performs an in-memory accumulate across all sources. Step 2 (`OP_WC_SYNC`): the completion tracker notifies every xPU that the accumulate is finished. Each xPU then reads back the reduced value.

The testbench in `tb/tb_fenghuang_tab.v` runs four directed scenarios covering each opcode and verifies that AllReduce built from `OP_WR_ACC` plus a trailing `OP_WC_SYNC` produces the expected

value. All four tests pass under `iverilog -g2005-sv`.

Synthesis target. The RTL is behavioural, but pipeline depths are chosen so that the *isolated* per-operation latencies fall close to the paper's ASIC figures (Table 1). Target: 250 MHz (4 ns/cycle), Xilinx Versal VMK180 on the Yale ECL board.

Table 1: Per-operation latency: paper's ASIC reference vs. our RTL estimated at 250 MHz. Cycle counts decompose roughly as issue + crossbar + bank-access + return.

Operation	Cycles	FPGA @ 250 MHz	Paper ASIC	Δ
`OP_READ`	55	220 ns	220 ns	0%
`OP_WRITE`	22	88 ns	90 ns	−2%
`OP_WR_ACC` *(isolated)*	22	88 ns	90 ns	−2%
`OP_WR_ACC` *(N=4)*	90	360 ns	—	—
`OP_WC_SYNC` *(lower bound)*	10	40 ns	40 ns	0%

Paper Table 3.1 reports only *isolated* per-op latency: (i) the 90 ns accumulate ignores bank serialisation, so a real N-xPU AllReduce costs $N \times 90$ ns (≈ 360 ns at N=4); (ii) the 40 ns `OP_WC_SYNC` is a register-stage lower bound that omits SerDes propagation and any deeper pipeline; (iii) the implied single-bit "write-issued" signal to the completion tracker undercounts concurrent grants by $N-1$, requiring a per-xPU counter we add in the RTL to keep the WC-Sync barrier correct.

Architectural Gaps and Failure Modes

Before we poceed to the redesign, this section takes a deeper architectural dive and concisely explains which assumptions in the paper *fail* under scrutiny, ordered from most to least severe.

xPU-TAB Interface. SerDes already lives on modern xPUs (NVLink uses 112G PAM4 for GPU-to-GPU serial communication), but FengHuang asks it to replace the memory bus, an order of magnitude harder. At today's state of the art, reaching 4.8 TB/s requires 4,800/28 $\approx$ 171 lanes per GPU, or $171 \times 4 = 684$ lanes on the TAB side. Adding LPDDR6 memory, power/ground, and control brings well above the pin ceiling of current large-die package technologies.

Signal Integrity. Even if such a package could be built, the high-speed SerDes signals must travel from the GPU to the TAB chip across PCB copper traces, typically 3-8 inches on a server motherboard. Copper traces are lossy transmission lines, and loss increases sharply with frequency.

MoE Prefetch. The paper presents MoE as the primary use case for the TAB, motivated by the trend that per-token active compute remains roughly constant while total parameter count grows (FengHuang Figure 2.6). Currently expert selection is determined by the router at *runtime*, taking the current hidden state as input. The routing decision cannot be known before the router runs, and the router runs on the output of the *previous* layer. The prefetcher therefore *cannot* issue the expert weight prefetch until after the current layer's attention and router have both completed.

Memory-Bandwidth Boundedness. The paper frames inference as uniformly memory-bandwidth-bound, appealing to the low arithmetic intensity of streaming model weights. The framing holds for low-batch *decode*. It does not hold for *prefill*, nor for the larger-batch decode regimes that production serving systems run today. Treating the entire pipeline as memory-bound therefore overstates the regime in which TAB's bandwidth provision is the load-bearing optimisation.

Continuous Batching. The TAB prefetcher is defined as per-layer and globally synchronised. Under continuous batching, issuing a prefetch for "layer N+1" is undefined, the scheduler must decide

which request's layer $N{+}1$ to prioritise, or prefetch multiple candidates, multiplying bandwidth demand. The paper's latency model assumes a monolithic batch advancing through layers in lockstep. Production serving systems have not operated this way since 2023.

PagedAttention. Every production inference framework (SGLang, vLLM) uses PagedAttention: KV-cache is partitioned into fixed-size pages (typically 16–32 tokens) managed by a pointer table, enabling non-contiguous allocation and zero-copy eviction. With TAB, the KV-cache spans both HBM and LPDDR6 based on recency and capacity. The CUDA attention kernel must track, for each page, which physical medium it resides in, and emit a different memory instruction accordingly. The paper describes TAB integration at the level of a memory allocator extension; the actual integration depth is comparable to writing a new attention backend for every supported framework.

TAB Hybrid Redesign

The simulation in Section 4 confirms that the prefetch model itself is sound. The piece that does not survive scrutiny is the *form factor*. We propose three changes (Figure 5) that preserve the architectural idea while staying inside what current packaging and software stacks can deliver:

Move the bandwidth-critical path inside the package. Replacing the board-level PHXLINK SerDes with a UCIe die-to-die link at 2-5 μm bump pitch eliminates the lane-count and signal-integrity walls. As a tradeoff, the TAB becomes a chiplet co-packaged with each xPU on a single 2.5D interposer, sharing the same substrate as the HBM stacks. LPDDR6 capacity scales by adding chiplet variants rather than redesigning the package. Per-rack pooling, the headline benefit of disaggregation, then moves to a coarser tier and is treated as a capacity backstop. However, such design requires xPU interface redesign and is not "full scalable" nor "vendor-augnostic."

Make prefetch MoE-aware. A natural fix for dynamic MoE resolution is to *speculate* expert selection rather than wait for it: a small auxiliary gate trained on layer $n{-}1$'s hidden state issues speculative top-K prefetches for layer $n{+}1$, with cancellation logic for mispredictions, so the overlap window is recovered at the cost of an oversubscribed bandwidth tax. We have a prototype [2] of this predictive-router scheme in development but no viable end-to-end results yet, so we cannot guarantee the viability of such a model. Unfortunately, prediction accuracy depends strongly on model architecture, and the accuracy-vs-bandwidth tradeoff warrants its own evaluation.

Integrate at the attention-backend layer. TAB residency must be visible to the kernel, not hidden behind a memory allocator, so PagedAttention can route page lookups to HBM or LPDDR6.

Conclusion

FengHuang represents a deep architectural shift: realising it in production would require coordinated changes across hardware (TAB silicon, packaging), system software (NCCL, allocator, attention kernels), and serving stacks. This is a cost industry will not absorb without an order-of-magnitude win. The pressure motivating it is nonetheless real and accelerating. As ASIC AI accelerators from new entrants push compute density faster than HBM can feed it, even batched prefill is shifting from compute-bound to memory-bandwidth-bound, making capacity-from-compute disaggregation a research direction the field cannot avoid. Lookahead-1 prefetch is unlikely to be the final answer for CXL-class tiered memory, but the open question is no longer *whether* to disaggregate, only *where* the leverage point sits. Is this in the schedulable static structure of transformer inference, in coherent KV-cache across tiers, or in finer-grained locality techniques.

Deliverables: Protocol RTL Implementation, Prefetch Simulator, Redesign Strategy.
GitHub Repository: https://github.com/anton-mel/fenghuang

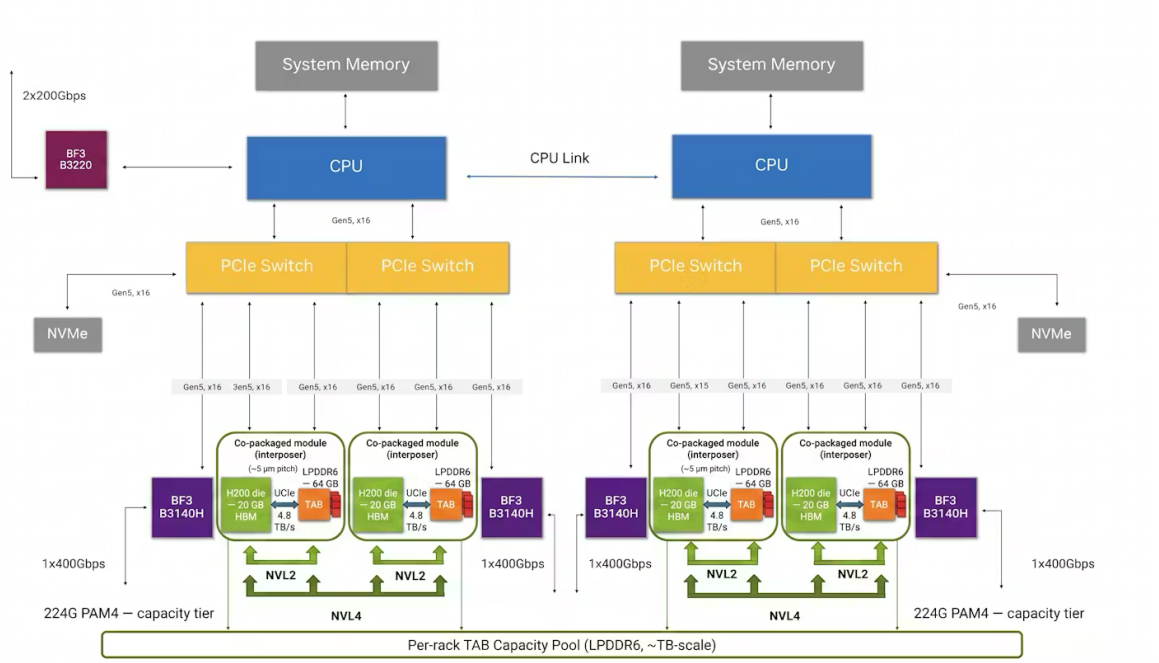


Figure 5: Hybrid TAB redesign. Each H200 is co-packaged with a TAB chiplet and an LPDDR6 stack over UCIe; a per-rack LPDDR6 capacity pool attaches via 224G PAM4 SerDes as a coarse capacity tier. The board-level interconnects (CPU, PCIe switches, NICs, NVMe) are unchanged.

References

[1] Microsoft Research. FengHuang: Next-Generation Memory Orchestration for AI Inferencing. arXiv:2511.10753v1, November 2025.

[2] A. Melnychuk. Source code, RTL, simulator, and benchmarks for this report. https://github.com/anton-mel/fenghuang, 2026.